GROUP XX1 (21)

1.Victor Gaiti -CT204/124293/25- Group Leader.

2.Robert Atuti -CT204/124360/25.

3.Faith Njeri -CT204/124301/25.

4.Trevor Lein -CT204/124264/25.

Fibonacci Series in Data Structures

A Case Study: Fibonacci-Based Customer Loyalty Rewards

1. Introduction and Objectives

The **Fibonacci sequence** starts with 0 and 1, and every next number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... In symbols, F(0) = 0, F(1) = 1 and F(n) = F(n − 1) + F(n − 2). This report uses F(0) = 0 everywhere.

The case study. A software company wants a loyalty reward system in which a customer in loyalty year y earns F(y) points. The system must be fast enough for thousands of customers and ready for future growth.

Objectives

- Generate Fibonacci numbers using recursion, iteration with a vector, and dynamic programming.
- Build a loyalty reward program that stores and displays yearly points.
- Compare the methods for n = 30 and recommend the best one.

Assumptions. The code is C++17 and uses only the standard library. Fibonacci rewards are a modelling choice for this case study, not a universal business rule. Because the sequence grows about 1.6 times each year, a real company would also set a maximum reward.

2. Methodology and C++ Implementations

(1) Fix one definition of F(n) and one validation policy.

(2) Implement each algorithm as a function returning `unsigned long long`.

(3) Check that all agree (F(10) = 55, F(30) = 832,040).

(4) Derive time and space complexity and exact operation counts.

Number type. The code uses `unsigned long long` (a 64-bit whole number). The largest Fibonacci number that fits is F(93) = 12,200,160,415,121,876,738. F(94) is too big and would silently give a wrong answer, so the programs only accept n from 0 to 93 (the loyalty program accepts 1 to 93).

Listing 1: fib_algorithms.hpp

```
#pragma once
#include <stdexcept>
#include <vector>

constexpr unsigned int MAX_INDEX = 93;   // largest n whose F(n) fits in 64 bits

// Task 1: recursion
unsigned long long fibRecursive(unsigned int n)
{
    if (n < 2) {
        return n;                          // F(0) = 0 and F(1) = 1
    }
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

// Tasks 2 and 3: builds F(0) ... F(n) in a vector (bottom-up table)
std::vector<unsigned long long> fibSeries(unsigned int n)
{
    if (n > MAX_INDEX) {
        throw std::invalid_argument("n is too large for 64-bit numbers");
    }
    std::vector<unsigned long long> series(n + 1);
    series[0] = 0;
    if (n >= 1) {
        series[1] = 1;
    }
    for (unsigned int i = 2; i <= n; ++i) {
        series[i] = series[i - 1] + series[i - 2];
    }
    return series;
}

// Extra: one value only, keeping just the last two numbers
unsigned long long fibTwoVar(unsigned int n)
{
    if (n == 0) {
```

```
        return 0;
    }
    unsigned long long previous = 0;      // F(0)
    unsigned long long current = 1;       // F(1)
    for (unsigned int i = 2; i <= n; ++i) {
        unsigned long long next = previous + current;
        previous = current;
        current = next;
    }
    return current;
}
```

2.1 Task One: Recursive approach

What it is. Recursion means a function solves a problem by calling itself on a smaller version of the same problem. For Fibonacci, F(n) calls F(n − 1) and F(n − 2).

How the code works. The *base cases* are F(0) = 0 and F(1) = 1. The line `if (n < 2) return n;` handles both, and it is what stops the function from calling itself forever.

Small example: F(4). F(4) = F(3) + F(2). F(3) = F(2) + F(1). F(2) = F(1) + F(0) = 1 + 0 = 1. So F(3) = 1 + 1 = 2 and F(4) = 2 + 1 = 3. This needs 9 function calls, and F(2) is worked out twice.

Complexity.

- **Time: O(2^n).** Each call makes up to two more calls, so the work roughly doubles each time n grows by one. (More precisely it grows like 1.618^n, but O(2^n) is the usual upper bound.)
- **Auxiliary space: O(n).** Up to n calls can be waiting on the call stack at once.

Why it becomes slow. The same small problems are solved again and again. The number of calls for F(n) is exactly 2·F(n + 1) − 1:

n	4	10	20	30	40
Calls	9	177	21,891	2,692,537	331,160,281

Increasing n by 10 multiplies the work by about 120. Nothing is saved between calls, so this method does not remember anything.

2.2 Task Two: Iterative approach using a vector

What it is. Iteration means repeating steps in a loop instead of calling the same function again.

Algorithm (the function `fibSeries`).

1. Reject n above 93.
2. Make a vector with n + 1 spaces (positions 0 to n).
3. Put 0 in position 0, and 1 in position 1 (if n is at least 1).
4. For i from 2 to n, set position i to the sum of positions i − 1 and i − 2.
5. Return the vector.

The loop stops at n, and the check `n >= 1` stops the program from touching position 1 when the vector only has position 0, so no position outside the vector is used.

Sample output for n = 10 (from the demo program below):

```
Series: 0 1 1 2 3 5 8 13 21 34 55
```

Demo program (demo.cpp)

```
#include <iostream>
#include "fib_algorithms.hpp"

int main()
{
    const unsigned int n = 10;
    std::cout << "Recursive F(10): " << fibRecursive(n) << std::endl;
    std::cout << "Series: ";
    for (unsigned long long value : fibSeries(n)) {
        std::cout << value << ' ';
    }
    std::cout << std::endl;
    return 0;
}
```

This prints `Recursive F(10): 55` and then the series above (with a space after the last number).

Complexity. Time is **O(n)** because there are n − 1 additions. Storing all terms needs **O(n) space**. If only the last two values are kept and the sequence is not stored (the function `fibTwoVar`), the extra memory is **O(1)**.

Compared with recursion. Every value is calculated once, so the time is linear instead of exponential, and there is no call stack to build up.

2.3 Task Three: Dynamic programming

What it is. Dynamic programming (DP) means saving answers that were already calculated so the same work is never repeated. Fibonacci is a good fit because there are only n + 1 different problems, F(0) to F(n), but plain recursion solves them millions of times.

There are two styles. **Tabulation** (bottom-up) fills a table from the smallest value upwards. **Memoization** (top-down) keeps the recursive code but stores each answer the first time it is found. This report implements tabulation.

Code. The function `fibSeries` in Listing 1 is the tabulation solution. The vector is the table, and each new entry reuses the two entries before it:

`series[i] = series[i - 1] + series[i - 2];`

Sample output. For n = 10 it gives the same series as in Section 2.2: `0 1 1 2 3 5 8 13 21 34 55`.

Complexity. O(n) time and O(n) space for the table.

Iteration versus dynamic programming. For Fibonacci they are almost the same thing here. The array method and tabulation follow the same bottom-up steps; "dynamic programming" just names the idea behind them: store results and reuse them.

Compared with recursion. For n = 30, recursion makes 2,692,537 calls, while the table needs only 29 additions.

3. Customer Loyalty Reward System

Indexing rule used. Loyalty year y earns F(y) points, with F(0) = 0 and F(1) = 1. So year 1 earns 1, year 2 earns 1, year 3 earns 2, year 4 earns 3, and so on. In the vector, position y holds the points for year y. Position 0 (value 0) is not a loyalty year and is never printed. This is a modelling choice for the case study.

How the program works.

1. It asks for the number of loyal years.
2. It checks that the input is a number from 1 to 93. The limit 93 matches the largest Fibonacci number that fits in `unsigned long long`.
3. It builds the rewards with `fibSeries(years)` and stores them in a vector.
4. It prints a table of year and reward points.

Listing 2: loyalty.cpp (uses fib_algorithms.hpp)

```
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
#include "fib_algorithms.hpp"

int main()
{
    int years = 0;
    std::cout << "Enter the number of loyal years (1-93): ";
    if (!(std::cin >> years)) {
        std::cout << "Error: please enter a whole number." << std::endl;
        return 1;
```

```
    }
    if (years < 1 || years > 93) {
        std::cout << "Error: years must be between 1 and 93." << std::endl;
        return 1;
    }

    std::vector<unsigned long long> rewards = fibSeries(years);

    std::cout << std::endl << std::left << std::setw(6) << "Year"
              << std::right << std::setw(16) << "Reward points" << std::endl;
    std::cout << std::string(22, '-') << std::endl;
    for (int year = 1; year <= years; ++year) {
        std::cout << std::left << std::setw(6) << year
                  << std::right << std::setw(16) << rewards[year] << std::endl;
    }
    return 0;
}
```

Plain-English notes. `std::cin >> years` reads the number, and it fails if the user types text. `setw` sets the column width so the table lines up. The loop starts at year 1 and ends at `years`, which is inside the vector, so no position is read outside it.

Sample run (input 5)

```
Enter the number of loyal years (1-93): 5

Year     Reward points
----------------------
1                     1
2                     1
3                     2
4                     3
5                     5
```

Input such as 0, −3 or 94 prints "Error: years must be between 1 and 93.", and text such as abc prints "Error: please enter a whole number." In both cases the program stops.

Large numbers. Ordinary integer types have a maximum size. If the program allowed 94 years, the points would wrap around to a wrong value without any warning. That is why the input limit is 93. A larger limit would need a special big-number type, and in real life a reward cap.

Many customers. For thousands of customers, store each customer's loyal years, call `fibSeries` **once** for the largest number of years, and look up `rewards[customerYears]` for each customer. That is one O(n) build plus a quick lookup per customer, instead of recalculating for everyone.

4. Performance Analysis for n = 30

All methods give the same answer, F(30) = 832,040. One difference matters: recursion and `fibTwoVar` calculate **one** value, while `fibSeries` calculates **and stores the whole series** F(0) to F(30). The benchmark in Listing 3 takes only the last value from `fibSeries`, so all methods answer the same question.

Listing 3: benchmark.cpp (uses fib_algorithms.hpp)

```
#include <chrono>
#include <iostream>
#include "fib_algorithms.hpp"

unsigned long long fibFromSeries(unsigned int n)
{
    return fibSeries(n).back();            // last value = F(n)
}

// Runs one method many times and returns the average time per call in nanoseconds
double timeMethod(unsigned long long (*method)(unsigned int),
                  int repetitions, unsigned long long& checksum)
{
    volatile unsigned int requestedN = 30; // stops the compiler pre-computing the
answer
    const auto start = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; ++i) {
        checksum += method(requestedN);
    }
    const std::chrono::duration<double, std::nano> elapsed =
        std::chrono::steady_clock::now() - start;
    return elapsed.count() / repetitions;
}

int main()
{
    if (fibRecursive(30) != 832040ULL || fibFromSeries(30) != 832040ULL ||
        fibTwoVar(30) != 832040ULL) {
        std::cerr << "A method returned a wrong value." << std::endl;
        return 1;
    }

    unsigned long long checksum = 0;       // printed so the work cannot be removed
    std::cout << "Direct recursion:          "
              << timeMethod(fibRecursive, 20, checksum) << " ns per call" <<
std::endl;
    std::cout << "Iterative array / table:   "
              << timeMethod(fibFromSeries, 200000, checksum) << " ns per call" <<
std::endl;
    std::cout << "Iterative two variables:   "
              << timeMethod(fibTwoVar, 200000, checksum) << " ns per call" <<
std::endl;
    std::cout << "Checksum: " << checksum << std::endl;
    return 0;
}
```

How to run it. Put the three files (fib_algorithms.hpp, benchmark.cpp) in one folder and run:

`g++ -std=c++17 -O2 -o benchmark benchmark.cpp` then `./benchmark`

Run it three times and record a typical result. Write down your computer, operating system and compiler version.

Results. No timings were measured for this draft, so the time column is pending. Do not invent values; fill the column in after running the program.

Approach	Time complexity	Auxiliary space	Main advantage	Main disadvantage	Measured time (n = 30)
Recursion	$O(2^n)$	O(n)	Simple, matches the definition	Repeats work; very slow as n grows	[MEASURE]
Iterative array / DP table	O(n)	O(n)	Fast; stores every value needed	Uses memory for the table	[MEASURE]
Two variables	O(n)	O(1)	Fast and tiny memory	Gives only one value, not the series	[MEASURE]

Operation counts (calculated, not measured). For n = 30, recursion makes 2,692,537 calls and about 1.3 million additions (1,346,268). The array and two-variable methods make 29 additions.

Discussion. Recursion is expected to be much slower than the linear methods because of this huge number of repeated calculations. I do not claim a fixed order between the two linear methods: their times depend on the computer, the compiler and memory use (the vector method has to create a vector each time). Very small times are also affected by timer noise. Big-O describes how work grows as n grows, while a measurement shows what one computer did for one value of n.

5. Recommendation and Conclusion

Recommendation. Use **iterative tabulation with a vector** (`fibSeries`) for the loyalty system.

- **Speed:** linear time, O(n).
- **Memory:** the table has at most 94 numbers, which is tiny.
- **Simplicity:** a short loop that is easy to read, test and maintain.
- **Many customers:** build the table once and look up each customer's points.
- **Stored output:** the reward for every year is kept, which the system needs.

The two-variable method is better when only one Fibonacci value is needed and the series does not have to be stored. Recursion is useful for learning but unsuitable for this system.

Fibonacci calculation is only one small part of the system. A real one also needs customer record storage (such as a database), input validation everywhere data is entered, and sensible reward limits.

Conclusion. Direct recursion is simple but exponential: it needs 2,692,537 calls for n = 30. Iteration and dynamic programming reduce this to linear time by calculating each value once. For the loyalty system, array-based tabulation gives the best balance of speed, clarity and stored results, as long as the years stay within 93 and the company sets a reward cap.

6. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data structures and algorithms*. Addison-Wesley.

cppreference.com. (n.d.). *C++ reference*. https://en.cppreference.com/